HOG_LBP_KAZE_SVM_RF_Goz_Kod_Aciklamasi.pdf - local repository text copy

Eye ROI - HOG + LBP + KAZE - SVM / Random Forest - Kod ve Yöntem Raporu
GÖZ BÖLGESİ DEEPFAKE TESPİTİ
HOG + LBP + KAZE -> RBF-SVM / Random Forest
Kod Yapısı ve Yöntem Açıklama Raporu
Alan Notebookta kullanılan yapı
Bölge Combined Eye ROI
Pozitif sınıf Fake = 1
Görüntü boyutu 224 x 224
Temel özellikler HOG + LBP
Ek lokal özellik KAZE
Sınıflandırıcılar RBF-SVM ve Random Forest
Model seçimi Validation F1, ROC-AUC tie-break
Test kullanımı Yalnız final değerlendirme
Seed 42
Çıktı kökü Kader / Deney 1 / Sonuçlar / <run_id>
Kaynak: HOG_LBP_KAZE_SVM_RF_Eye_Deepfake_Deney.ipynb içindeki kod ve açıklamalar.
Eye ROI - HOG + LBP + KAZE - SVM / Random Forest - Kod ve Yöntem Raporu
1. Raporun Amacı
Bu rapor, notebook'un yalnızca ne yaptığını değil, neden bu yöntemlerin bir arada kullanıldığını ve kodun deney
güvenliği açısından nasıl yapılandırıldığını açıklar. Deney, derin bir sinir ağı eğitmek yerine göz bölgesinden elle
tasarlanmış özellikler çıkaran klasik bir makine öğrenmesi hattıdır. Temel hedef; göz ROI'sindeki şekil, mikro-doku
ve lokal anahtar nokta bilgisinin Real/Fake ayrımına ne ölçüde katkı sağladığını üç farklı model ailesiyle
karşılaştırmaktır.
Şekil 1. Notebook'un uçtan uca deney akışı.
2. Deney Tasarımının Mantığı
Neden tek model değil? Notebook üç önceden tanımlı model ailesini aynı train/validation/test yapısında
karşılaştırır. Böylece KAZE'nin ek katkısı ve sınıflandırıcı türünün etkisi birbirinden ayrılabilir.

Model ailesi	Kullanılan özellik	Sınıflandırıcı	Deneydeki rol
Model A - Baseline	HOG + LBP	RBF-SVM	KAZE eklenmeden klasik şekil + doku baseline'ı
Model B - Feature Fusion	HOG + LBP + KAZE	RBF-SVM	Lokal keypoint/deskriptor bilgisinin SVM'e katkısını ölçmek
Model C - Alternatif karar modeli	HOG + LBP + KAZE	Random Forest	Aynı fused özelliklerde ağaç tabanlı karar mekanizmasını karşılaştırmak

3. Notebook Hücre Yapısı

Bölüm	Görev	Neden gerekli?
Ön hazırlık hücreleri	OpenCV/KAZE kontrolü, contrib kurulumu, runtime restart	KAZE_create desteğini garanti etmek
Cell 1	Bağımlılıklar, importlar, seed	Tekrar üretilebilir ve eksiksiz çalışma ortamı
Cell 2	local source package ve strict path resolution	Yanlış klasörden veri okuma riskini azaltmak
Cell 3	YAML config, Run ID, çıktı klasörleri	Deney ayarlarını koddan ayırmak ve run bazlı kayıt
Cell 4	Logging, atomik I/O, hashing, environment lock	Bozuk/yarım dosya ve izlenebilirlik sorunlarını azaltmak
Cell 5	Metadata, accounting, leakage gates	Veri kalitesini ve split güvenliğini doğrulamak
Cell 6	HOG/LBP/KAZE fonksiyonları	Sabit ve denetlenebilir feature extraction
Cell 7	Classical ML smoke test	Uzun çalışmadan önce pipeline'ı 4 örnekle sınamak
Cell 8	Feature extraction + cache	Tekrar hesaplamayı önlemek, kesintiden sonra hız kazanmak
Cell 9	Train-only scaling + metrics	Leakage'siz ölçekleme ve ortak metrik fonksiyonları
Cell 10	Validation-only model seçimi	Test setine dokunmadan adayları karşılaştırmak
Cell 11	Atomik model paketi + fresh-load test	Kaydedilen modelin gerçekten yeniden çalıştığını kanıtlamak
Cell 12	Final test	Üç finalize modelin aynı testte objektif karşılaştırması
Cell 13	Group-level aggregation	Metadata video_id grupları için yardımcı

Eye ROI - HOG + LBP + KAZE - SVM / Random Forest - Kod ve Yöntem Raporu
analiz
Cell 14 Rapor görselleri Okunabilir, karşılaştırılabilir çıktılar
Cell 15 Run summary + manifest + final gate Eksiksiz deney paketini doğrulayıp kapatmak
4. Girdi Verisi: Neden Combined Eye ROI?
Notebook doğrudan önceden çıkarılmış combined-eye ROI görüntülerini kullanır; yüzün tamamını veya ağız/kaş gibi
başka bölgeleri yeniden üretmez. Böylece deneyin inceleme alanı yalnız göz bölgesiyle sınırlandırılır ve HOG, LBP,
KAZE gibi klasik özelliklerin göz bölgesindeki sinyale odaklanması sağlanır.
Sınıf kodlaması: `real = 0`, `fake = 1`. Precision, Recall, F1 ve threshold araması bu nedenle Fake sınıfını pozitif
sınıf kabul eder.
Split politikası: Train/Validation/Test mevcut metadata'dan okunur. Notebook yeni rastgele split oluşturmaz; bu,
daha önce tanımlanmış deney bölünmelerini korur.
5. Hiperparametreler ve Neden Bu Değerler?
Parametre Değer Kod içindeki işlevi / teknik gerekçe
Image size 224 x 224 Bütün ROI'leri aynı geometriye getirerek feature boyutunu sabitler
HOG orientations 9 Gradyan yönlerini dokuz histogram yönünde özetler
HOG pixels_per_cell 16 x 16 224 px görüntüyü 14 x 14 hücreye böler
HOG cells_per_block 2 x 2 Komşu hücreleri birlikte normalize ederek lokal kontrast dayanıklılığı sağlar
HOG block norm L2-Hys HOG'da yaygın kontrast normalizasyonu
HOG transform_sqrt True Yoğunluk/kontrast etkisini bastırmaya yardımcı olur
LBP radius 3 Merkez piksel çevresinde daha geniş lokal doku komşuluğu
LBP points 24 24 komşu örnekle daha ayrıntılı mikro-doku kodlaması
LBP method uniform Daha kompakt ve kararlı LBP histogramı
KAZE descriptor 64 OpenCV KAZE descriptor uzunluğu; mean+std ile 128 boyuta dönüşür
SVM C 0.1, 1, 10 Düzenlileştirme ile training uyumu arasındaki üç farklı sertlik düzeyi
SVM kernel RBF Doğrusal olmayan sınıf sınırlarını modellemek
SVM gamma scale Feature varyansına göre otomatik RBF ölçeği
SVM class_weight balanced Sınıf sayılarından gelen olası dengesizliği otomatik telafi etmek
RF n_estimators 300 Kararlı ensemble tahmini için çok sayıda ağaç
RF max_features sqrt Ağaçlar arası çeşitliliği artırmak
Selection metric F1 Fake precision ve recall'u birlikte dikkate almak
SHA256 workers 8 local source dosyalarının hash kontrolünü paralelleştirmek
Figure DPI 150 Rapor görsellerinde yeterli çözünürlük
Min short edge 600 px Okunamayacak kadar küçük grafik üretimini hard-fail yapmak
6. HOG: Göz Bölgesinde Neyi Temsil Ediyor?
HOG (Histogram of Oriented Gradients), piksel yoğunluklarından doğrudan sınıf öğrenmek yerine lokal gradyanların
yönünü ve büyüklüğünü özetler. Göz ROI'sinde bu temsil; göz kapağı sınırı, göz köşeleri, kirpik çizgileri ve lokal
geometrik kenarlar gibi yapısal bilgileri kodlamak için uygundur.
Eye ROI - HOG + LBP + KAZE - SVM / Random Forest - Kod ve Yöntem Raporu
224 x 224 görüntü, 16 x 16 hücrelerle 14 x 14 hücreye ayrılır. 2 x 2 blok, 9 yönelim kullanımında beklenen HOG
boyutu 13 x 13 x 4 x 9 = 6,084 özelliktir.
7. LBP: Neden HOG'un Yanına Ekleniyor?
LBP (Local Binary Pattern), HOG'un geometrik/yönelim bilgisini tamamlayan bir mikro-doku tanımlayıcısıdır. Merkez
piksel ile çevresindeki komşuların göreli yoğunluk ilişkisini kodlar. Notebook P=24, R=3 ve uniform yöntemi kullanır.
Uniform LBP için histogram boyutu P+2 olduğundan 26 özellik oluşur.

HOG + LBP baseline boyutu: 6,084 + 26 = 6,110 özellik. Bu model KAZE olmadan elde edilen temel klasik ML referansıdır.

8. KAZE: Neden Kullanılıyor ve Nasıl Sabit Boyuta Getiriliyor?

KAZE, görüntüyü nonlinear scale-space içinde inceleyen lokal keypoint/deskriptor tabanlı bir yöntemdir. HOG ve LBP tüm görüntüyü düzenli grid üzerinden özetlerken KAZE, belirgin lokal noktaları ve bu noktaların çevresindeki yapıyı temsil eder. Bu nedenle KAZE'nin eklenmesi, global/grid tabanlı texture bilgisinin kaçırabileceği lokal göz artefaktlarını yakalamayı amaçlar.

Bir görüntüde bulunan KAZE keypoint sayısı sabit değildir. SVM veya Random Forest ise her örnek için aynı uzunlukta vektör ister. Notebook bu sorunu descriptor'ların her boyutu için ortalama ve standart sapmayı alarak çözer. KAZE descriptor uzunluğu 64 olduğu için 64 mean + 64 std = 128 boyutlu sabit bir KAZE vektörü elde edilir.

Fused özellik boyutu: 6,110 (HOG+LBP) + 128 (KAZE) = 6,238 özellik.

Bir görüntüde hiç KAZE descriptor bulunamazsa kod 128 uzunluklu sıfır vektörü döndürür. Bu, örneği silmek yerine feature boyutunu korur ve pipeline'ın deterministik devam etmesini sağlar.

9. Neden RBF-SVM?

Notebook HOG/LBP/KAZE gibi yüksek boyutlu sabit feature vektörlerini klasik bir sınıflandırıcıya verir. RBF-SVM, sınıflar arasındaki ilişkinin doğrusal bir hiper-düzlemle ayrılamadığı durumlarda nonlinear karar sınırı kurabilir. Bu nedenle hem HOG+LBP baseline hem de KAZE eklenmiş fused temsil aynı RBF-SVM yapısıyla denenir.

C Yorum

0.1 Daha güçlü düzenlileştirme; daha yumuşak karar sınırı

1.0 Orta düzey regularization/fit dengesi

10.0 Training örneklerine daha sıkı uyum; daha karmaşık sınır ihtimali

`gamma='scale'`, RBF'nin etki yarıçapını feature sayısı ve varyansa göre otomatik belirler.

`class_weight='balanced'` ise sınıf ağırlıklarını örnek sayısına göre ayarlayarak sınıf dengesizliğinin karar sınırına etkisini azaltmayı hedefler.

10. Neden Random Forest da Karşılaştırılıyor?

Random Forest, aynı HOG+LBP+KAZE vektöründe SVM'den farklı bir karar mekanizması sunar. Ağaç tabanlı model feature ölçeğine SVM kadar duyarlı değildir; bu nedenle RF doğrudan ölçeklenmemiş fused vektörlerle eğitilir. Böylece aynı feature setinin kernel tabanlı SVM ve ensemble-ağaç yaklaşımındaki davranışı kıyaslanabilir.

RF adayı	n_estimators	max_depth	min_samples_leaf	max_features	Amaç
RF-1	300	None	1	sqrt	Tam derinlikte temel güçlü RF
RF-2	300	20	1	sqrt	Derinliği sınırlandırıp overfit riskini
Eye ROI - HOG + LBP + KAZE - SVM / Random Forest - Kod ve Yöntem Raporu					azaltmak
RF-3	300	None	2	sqrt	Leaf başına minimum örneği artırarak daha yumuşak ağaçlar

11. StandardScaler Neden Sadece Train Üzerinde Fit Ediliyor?

HOG, LBP ve KAZE bileşenleri farklı sayısal dağılımlara sahiptir. SVM mesafeye dayalı kernel kullandığı için feature ölçeği önemlidir. Notebook iki ayrı scaler oluşturur: HOG+LBP ve HOG+LBP+KAZE. Her scaler yalnız train verisinde `fit_transform` görür; validation yalnız `transform` edilir. Test ise model seçimi tamamlanana kadar hiç dönüştürülmez.

Neden önemli? Validation veya test istatistikleri scaler fit'ine girerse veri sızıntısı oluşur. Kod bu riski bilinçli olarak engeller.

12. Model ve Threshold Seçimi

SVM adaylarında her C değeri train setinde fit edilir; validation `decision_function` skoru üzerinde Precision-Recall eğrisi oluşturulur ve F1'i maksimize eden threshold seçilir. Random Forest için aynı mantık `predict_proba[:,1]` skorunda uygulanır.

Birincil seçim: Validation F1. Kod `score = (F1, ROC-AUC)` tuple'ı kullandığından F1 eşitliğinde ROC-AUC tie-
break olarak devreye girer.
Neden threshold 0 veya 0.5 değil? Varsayılan karar eşiği veri setinin precision/recall dengesini en iyi noktaya
getirmek zorunda değildir. Validation üzerinden threshold seçmek, Fake sınıfı için F1'i deneysel olarak optimize
eder.
13. Test Seti İzolasyonu
Notebook model ailesi, SVM C değeri, RF konfigürasyonu ve threshold seçimi boyunca test setini kullanmaz. Test
feature'larının scaler transform'u bile final aşamaya kadar ertelenmiştir. Cell 12'de üç model validation ile finalize
edildikten sonra aynı sabit test setinde karşılaştırılır.
Metrik	Kodda hesaplanan anlam
Accuracy	Toplam doğru sınıflandırma oranı
Balanced Accuracy	Sınıf başına recall ortalaması
Precision	FAKE denilen örneklerin ne kadarı gerçekten FAKE
Recall / Sensitivity	Gerçek FAKE örneklerin ne kadarı yakalandı
F1	Precision ve Recall harmonik ortalaması
Specificity	Gerçek REAL örneklerin ne kadarı doğru REAL bulundu
ROC-AUC	Threshold bağımsız skor sıralama gücü
Average Precision	Precision-Recall davranışının özet ölçümü

14. Metadata, Data Accounting ve Leakage Quality Gate'leri
Notebook `sample_id`, `source_frame`, `relative_frame_path`, `label`, `split`, `video_id`, `frame_stem`, `face_id`,
`combined_eye_path` ve `status` alanlarını zorunlu tutar. SUCCESS kayıtlarında sample_id boş veya duplicate
olamaz. SKIPPED/no_face kayıtlarında boş kimlik varsa yalnız denetim amacıyla deterministic audit ID üretilir.
Kontrol	Kodun davranışı
Dosya varlığı	SUCCESS combined-eye yolları fiziksel olarak mevcut olmalı
Frame/face index	SUCCESS örneklerde çözülemeyen indeks hard-fail
Eye ROI - HOG + LBP + KAZE - SVM / Random Forest - Kod ve Yöntem Raporu	
Class presence	Train/Val/Test'in her birinde REAL ve FAKE bulunmalı
Path leakage	Aynı output_path birden fazla splitte olamaz
SHA256 leakage	Aynı dosya içeriği farklı splitteyse deney durur
video_id group overlap	Mevcut video_id grupları splitler arası kesişemez
True video leakage	video_id gerçek source-video kanıtı değilse PASS uydurulmaz

SHA256 paralel `ThreadPoolExecutor` ile sekiz worker üzerinden hesaplanır. Bu seçim özellikle local source package
üzerindeki çok sayıda küçük dosyanın seri okunmasında zaman maliyetini düşürmeyi amaçlar.
15. Smoke Test ve Sayısal Güvenlik
Uzun feature extraction başlamadan önce train setinden 2 REAL + 2 FAKE örnek seçilir. Bu dört örnekte HOG/LBP/KAZE çıkarımı, boyut tutarlılığı, StandardScaler ve Linear SVM çalıştırılır. Amaç nihai model seçmek
değil, pipeline'ın temel olarak çalıştığını erken doğrulamaktır.
NaN/Inf koruması: HOG, LBP, KAZE, base ve fused vektörlerin tümünde `np.isfinite` kontrolü yapılır. Sayısal
bozulma görülürse deney sessizce devam etmez.
16. Feature Cache Neden Var?
HOG/LBP/KAZE çıkarımı, özellikle binlerce local source görüntüsünde tekrar tekrar çalıştırılması pahalı bir adımdır.
Notebook train/val/test feature matrislerini ayrı sıkıştırılmış NPZ dosyalarında saklar. Cache mevcutsa sample_id
sırası metadata ile doğrulanır ve ancak uyuşuyorsa tekrar kullanılır.
Kazanç: Colab kesintisi veya aynı run'da sonraki hücreleri yeniden çalıştırma durumunda feature extraction tekrar
yapılmaz.
17. Atomik Kayıt ve Fresh-Load Doğrulaması
JSON, CSV, joblib ve NPZ dosyaları önce geçici dosyaya yazılır, mümkün olduğunda tekrar okunarak bütünlük

kontrolü yapılır ve sonrasında `os.replace` ile hedef dosyaya geçirilir. Bu yaklaşım, çalışma ortasında kesinti olursa
yarım yazılmış dosyanın final çıktı gibi görünmesi riskini azaltır.
Üç model paketi model, scaler (RF için None), validation-selected threshold, config ve RNG state bilgisiyle joblib
olarak kaydedilir. Ardından dosya sıfırdan yüklenir ve validation skorlarının orijinal model skorlarıyla `np.allclose`

düzeyinde aynı olduğu doğrulanır.
18. Group-Level Analiz Neden 'Video-Level' Denmiyor?
Cell 13, test tahminlerini `video_id` ve label ile gruplayıp ortalama skor üretir. Ancak notebook açıkça bu alanın
authoritative original source-video kimliğini kanıtlamayabileceğini kabul eder. Bu nedenle çıktı `metadata_group_level_metrics.csv` olarak adlandırılır ve gerçek video-level performans iddiası yapılmaz.
Bilimsel gerekçe: Yanlış bir video-level PASS veya genelleme iddiası üretmek yerine metadata'nın desteklediği
anlam düzeyi korunur.
19. Grafik ve Raporlama Standardı
Grafik Ne gösterir?
01_dataset_distribution Train/Val/Test REAL-FAKE dağılımı
02_validation_f1_candidates Tüm model adaylarının validation F1 karşılaştırması
03_final_test_metrics Üç modelin test metriklerini aynı grafikte karşılaştırma
04-06 confusion matrices Her model için TN/FP/FN/TP dağılımı
Eye ROI - HOG + LBP + KAZE - SVM / Random Forest - Kod ve Yöntem Raporu
07_roc_curves Üç modelin threshold-bağımsız ROC performansı
08_precision_recall_curves FAKE sınıfında precision-recall dengesi
09_score_distributions REAL/FAKE skor dağılımlarının ayrışması
Tüm grafikler 150 DPI ile oluşturulur ve kısa kenar 600 pikselin altındaysa quality gate hata verir.
Başlık ve eksenler
İngilizcedir; bu, farklı deney raporlarında görsel standardı korur.
20. Çıktı Klasörü ve Deney İzlenebilirliği
Klasör / dosya İçerik
checkpoints/ Üç finalize model paketi (.joblib)
logs/run.log Zaman damgalı çalışma kayıtları
metrics/ Accounting, validation search, final test, group metrics, quality gates
predictions/ Örnek bazlı ve group-level tahmin tabloları
figures/ Dokuz rapor grafiği
artifacts/ Metadata used, feature caches, feature dimensions
config_resolved.yaml O run'da gerçekten kullanılan ayarlar
requirements_lock.txt Python paket sürümleri
environment.json Python/OpenCV/NumPy/Pandas ortam bilgisi
output_manifest.csv Final dosyaların boyut ve SHA256 manifesti
run_summary.json Deneyin tek dosyada özet kaydı
Run ID tarih-saat + bölge + yöntem + seed bilgisini içerir. Aynı dakika içinde klasör zaten varsa `_r2`, `_r3` gibi
suffix eklenir; böylece önceki deney üzerine yazılmaz.
21. Kodun Teknik Olarak Önemli Notları
OpenCV / KAZE kurulumu: Notebook'un başındaki ayrı hücreler OpenCV contrib paketini kurup runtime'ı yeniden
başlatır. Bunun nedeni bazı Colab OpenCV kurulumlarında `KAZE_create` fonksiyonunun bulunmamasıdır.
`selection_metric` config alanı: Config'te `f1` olarak kayıtlıdır; uygulama da doğrudan F1 threshold araması ve F1
tabanlı model seçimi yapar. Bu sürümde metrik dinamik olarak başka bir değere çevrilmez.
`verify_images` config alanı: Config içinde tanımlıdır ancak ayrı bir preflight `PIL.verify()` döngüsü görünmez.
Görüntü okunabilirliği feature extraction sırasında `cv2.imread` ile fiilen kontrol edilir.
Klasik ML checkpoint semantiği: Bu deneyde epoch/optimizer/scheduler state'i yoktur. Kod bunu PyTorch checkpoint'i gibi göstermek yerine SVC/RF + scaler + threshold + config + RNG state paketi olarak kaydeder.
22. Yöntemlerin Birlikte Kullanılmasının Özeti
Bileşen Yakaladığı bilgi Diğer bileşeni nasıl tamamlıyor?
HOG Kenar ve yönelim LBP'nin doku bilgisinin geometrik bağlamını verir
LBP Lokal mikro-doku HOG'un kaçırdığı küçük yoğunluk örüntülerini ekler
KAZE Lokal keypoint/deskriptor Grid tabanlı HOG/LBP dışındaki seçici lokal bölgeleri ekler
RBF-SVM Nonlinear karar sınırı Yüksek boyutlu standartlaştırılmış feature uzayını ayırır
Random Forest Ağaç tabanlı nonlinear karar Aynı fused feature için alternatif model davranışı sunar
Eye ROI - HOG + LBP + KAZE - SVM / Random Forest - Kod ve Yöntem Raporu
23. Genel Değerlendirme
Notebook, göz bölgesinde klasik görüntü işleme ve makine öğrenmesini kontrollü bir ablation mantığıyla birleştirir.
HOG+LBP modeli temel baseline'dır; KAZE eklenerek lokal anahtar nokta bilgisinin katkısı ölçülür; aynı fused

özellik hem RBF-SVM hem Random Forest ile test edilerek feature etkisi ile classifier etkisi birbirinden ayrılmaya
çalışılır.
Kodun güçlü deneysel yönleri; mevcut splitleri koruması, train-only scaling, validation-only model/threshold seçimi,
test izolasyonu, SHA256 exact-content leakage kontrolü, feature cache, atomik kayıt, fresh-load doğrulaması ve
metadata'nın izin vermediği gerçek video-level iddiayı üretmemesidir.
Bu rapor sonuç metriklerini yorumlayan bir deney sonuç raporu değildir; notebook'un yöntemini, seçimlerinin teknik
gerekçesini ve kod mimarisini açıklayan kod/yöntem raporudur. Nihai metrikler notebook çalıştırıldıktan sonra
oluşan `final_test_metrics.csv` ve ilgili grafiklerden ayrıca sonuç raporuna aktarılmalıdır.